Arquitectura de los modelos gray-box de Wilson-Cowan

Repositorio: `PedroCappelletti/wilson-cowan-id`. Las rutas de archivos y funciones son relativas a la raíz del repositorio.

1. Planta, datos y notación

Los modelos se ajustan sobre trayectorias simuladas. El estado observado es $x = (I, E)^\top$, las actividades de las poblaciones inhibitoria y excitatoria. La entrada es $u = (P, Q)^\top$, el comando de luz sobre la población excitatoria (P) y sobre la inhibitoria (Q). La salida del modelo es la trayectoria completa de x; el ajuste no se hace contra el LFP.

Cada escenario dura 2 s, muestreado con $\Delta t = 0{,}5$ ms (4001 muestras), con el estímulo encendido entre 100 y 1900 ms. Las plantas son Wilson-Cowan con los parámetros verdaderos más un mecanismo que el modelo no tiene:

- **Refractariedad:** la sigmoidea se multiplica por $(1 - r\,x)$, con $r = 0{,}10$.
- **Actuador:** a las poblaciones no llega P sino $P_{ef} = 3\tanh(P_{lag}/3)$, con $\dot{P}_{lag} = (P - P_{lag})/\tau$ y $\tau = 1$ ms, y lo mismo para Q.

El comando se mantiene constante entre muestras, tanto en la planta como en el modelo. La planta se integra con RK4 a 0,05 ms, con 10 subpasos por muestra (`scripts/gen_uncertain_dataset.py`, `generar_con`). A I y E se les suma ruido blanco gaussiano independiente, con σ por muestra de 0,0032 y 0,016 (`scripts/esc_gen_ruido.py`).

Los estímulos de entrenamiento cubren $P \in [0;\ 2{,}0]$ y $Q \in [0;\ 1{,}5]$. Fuera de ese rango, el modelo extrapola.

2. Backbone

Todos los modelos comparten el campo de Wilson-Cowan reducido,

$$f_{WC}(x, u; \beta) = \begin{bmatrix} \frac{1}{\tau_i}(-I + \sigma(a_i u_i) - k_i) \\ \frac{1}{\tau_e}(-E + \sigma(a_e u_e) - k_e) \end{bmatrix}, \qquad \begin{aligned} u_i &= w_{IE}E - w_{II}I + Q - \theta_i \\ u_e &= w_{EE}E - w_{EI}I + P - \theta_e \end{aligned} \tag{1}$$

con $\sigma(z) = 1/(1 + e^{-z})$ y $\beta = (w_{EE}, w_{EI}, w_{IE}, w_{II}, \tau_e, \tau_i, a_e, a_i, \theta_e, \theta_i)$.

Los diez parámetros se guardan como variables libres $\tilde{\beta}$ y se transforman con $\beta = \text{softplus}(\tilde{\beta})$, para que sean siempre positivos. Los offsets no son libres: se recalculan en cada evaluación como $k_e = \sigma(-a_e\theta_e)$ y $k_i = \sigma(-a_i\theta_i)$, de modo que $x = 0$ con $u = 0$ sigue siendo un equilibrio cuando cambian a y θ.

P y Q entran sumados al argumento de cada sigmoidea, y en ningún otro lugar. Esa es la estructura que usa el despeje del controlador de Martínez et al. (2022, ec. 6).

Código: `GrayBoxWC.backbone(x, P, Q)`, en `src/neural_ode/dynamics.py`. Devuelve $(\dot{I}, \dot{E})$ en ese orden; la convención del estado en todo el código es `x[..., 0] = I`, `x[..., 1] = E`.

3. Modelo completo y discretización

Los modelos con corrección son aditivos:

$$\dot{x} = f_{WC}(x, u; \beta) + g_\phi(\xi) \tag{2}$$

donde ξ es la entrada de la corrección, que cambia según la variante (sección 4), y g_ϕ devuelve (g_I, g_E). Sin corrección, $g_\phi \equiv 0$ y el modelo es el white-box.

El modelo se integra con RK4 de paso fijo Δt, con el comando retenido durante el paso:

$$x_{k+1} = x_k + \tfrac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4), \qquad k_1 = F(x_k, u_k),\ \ k_2 = F(x_k + \tfrac{\Delta t}{2}k_1, u_k),\ \ldots \tag{3}$$

con F el lado derecho de (2). Código: `rk4_step` y `rollout`, en `src/neural_ode/integrate.py`. `rollout(f, x0, P_seq, Q_seq, dt)` recibe el estado inicial y la secuencia completa del comando, y devuelve la trayectoria de $T + 1$ puntos.

Las variantes con historia reciben en cada paso un vector con el comando actual y los retardos, no un escalar. El backbone toma siempre el canal 0, que es el comando actual.

4. Correcciones con red neuronal

En las tres variantes, g_ϕ es un perceptrón de dos capas ocultas de 32 unidades con tangente hiperbólica:

$$g_\phi(\xi) = W_3 \tanh\big(W_2 \tanh(W_1 \xi + b_1) + b_2\big) + b_3, \qquad W_3 \in \mathbb{R}^{2\times 32} \tag{4}$$

Los sesgos y W_3 arrancan en cero, así que al inicio del entrenamiento $g_\phi = 0$ y el modelo coincide con el backbone. Las variantes cambian solo ξ. Se elige con el argumento `correction_inputs` de `GrayBoxWC`; la red está en `GrayBoxWC.g` y se evalúa con `GrayBoxWC.g_out`.

4.1 Agnóstica: $\xi = (I, E)$

$\xi \in \mathbb{R}^2$ (variante `B`, `correction_inputs="x"`). La corrección depende solo del estado actual: no ve el comando ni el pasado. Tiene 1218 pesos, más los 10 de β.

Como g_ϕ no depende de u, (2) sigue siendo afín en u dentro de cada sigmoidea, y el despeje cerrado del controlador sigue valiendo: alcanza con restar $\tau\, g_\phi(x)$ del objetivo antes de invertir la sigmoidea.

4.2 Historia del comando: $\xi = (I, E, P_k, \dots, P_{k-K+1}, Q_k, \dots, Q_{k-K+1})$

$\xi \in \mathbb{R}^{2+2K}$ (variante `H`, `correction_inputs="xhist"`). La red ve el comando actual y los $K-1$ anteriores de cada canal, a paso Δt:

- con $K = 10$ (5 ms de memoria), 1858 pesos;
- con $K = 40$ (20 ms), 3778.

La historia se arma con `apilar_historia` (`src/neural_ode/graybox_train.py`), con ceros antes del inicio de la trayectoria. El orden de cada canal es $[u_k, u_{k-1}, \dots]$.

El comando actual u_k entra dos veces: en el argumento de la sigmoidea, por (1), y en la red, por (4). El modelo deja de ser afín en u_k.

4.3 Filtro entrenable: $\xi = (I, E, z)$, con $z = A\, h_k$

$h_k = (P_k, \dots, P_{k-K+1}, Q_k, \dots, Q_{k-K+1}) \in \mathbb{R}^{2K}$ es la misma historia de 4.2. Una capa lineal sin sesgo la resume en n canales:

$$z = A\, h_k, \qquad A \in \mathbb{R}^{n\times 2K}, \qquad \xi = (I, E, z) \in \mathbb{R}^{2+n} \tag{5}$$

Es la variante `K`, con `correction_inputs="xconv"` y `n_fir` $= n$. Cada fila de A es un filtro FIR sobre la historia de los dos comandos. A y la red se entrenan juntas, sin forma impuesta. Con $K = 40$, se probaron n = 4, 8 y 16: 1666, 2114 y 3010 pesos.

Código: `_FIRMLP` en `src/neural_ode/dynamics.py`. `_FIRMLP.fir` es A y `_FIRMLP.mlp` es (4).

z es lineal en u_k, pero la red no, así que como en 4.2 el comando actual entra en la sigmoidea y en la red. El rango efectivo de A queda bajo. Con los datos de 200 ms (muestreo de 20 kHz y $K = 400$), su dimensión efectiva fue de 2,7 con $n = 4$ y 3,9 con $n = 8$, y muestras vecinas del comando tenían correlación 0,990. La forma de cada fila de A no se puede leer como un núcleo físico.

5. Backbone expandido (sin red)

5.1 Forma exacta

Variante S (`structured=True`). Se agregan tres parámetros al backbone:

$$\dot{x}_j = \tfrac{1}{\tau_j}\Big(-x_j + (1 - r_j\, x_j)\, \sigma(a_j u_j) - k_j \Big), \qquad P \to \frac{\tanh(\alpha P)}{\alpha}, \quad Q \to \frac{\tanh(\alpha Q)}{\alpha} \tag{6}$$

con $r_j \in [0, 2]$ y $\alpha \in [10^{-3}, 10]$ acotados por recorte, sin softplus, para que el gradiente no se anule cerca de cero. $\alpha \to 0$ es el canal lineal. Son 13 parámetros. Con r arrancando cerca de 0, r_j queda trabado en el borde y el modelo colapsa al white-box; por eso se entrena arrancando del β del white-box y con $r_j = 0{,}05$.

En la planta con refractariedad, el modelo con σ = 0,01 identificó $r_E = 0{,}098$ (verdadero 0,10), $r_I = 0$ (verdadero 0,10) y α en la cota inferior, que es lo correcto, porque esa planta no satura.

El despeje sigue siendo cerrado: se divide el objetivo por $(1 - r_j x_j)$ y se invierte la saturación con artanh. Ya está implementado en `IMCController` (argumento `structured`).

5.2 Estado de filtro

`LagGrayBox`, en `src/neural_ode/memory.py`. El estado se aumenta con una copia del filtro del actuador:

$$y = (I, E, \hat{P}, \hat{Q})^\top, \qquad \dot{\hat{P}} = \frac{P - \hat{P}}{\hat{\tau}}, \quad \dot{\hat{Q}} = \frac{Q - \hat{Q}}{\hat{\tau}}, \quad \dot{x} = f_{WC}(x,\, s(\hat{P}),\, s(\hat{Q});\, \beta), \quad s(v) = \frac{\tanh(\alpha v)}{\alpha} \tag{7}$$

Son 12 parámetros: β, $\hat{\tau} = \text{softplus}(\tilde{\tau}) + \tau_{min}$ y α. El piso es $\tau_{min} = \text{máx}(0{,}15;\ 0{,}6\,\Delta t) = 0{,}3$ ms, porque con $\Delta t = 0{,}5$ ms y $\hat{\tau} < 0{,}18$ ms el RK4 se vuelve inestable.

$\hat{P}$ no se observa. Al inicio de cada ventana de entrenamiento se calcula exacto recorriendo el filtro sobre todo el comando, con la solución del retenedor de orden cero:

$$\hat{P}_{k+1} = a\, \hat{P}_k + (1 - a)\, P_k, \qquad a = e^{-\Delta t/\hat{\tau}}$$

Código: `LagGrayBox.filtered_inputs`. Sin esto, arrancar cada ventana con $\hat{P} = P$ metía un transitorio falso que sesgaba $\hat{\tau}$ a la mitad.

En la planta con actuador, el modelo identificó $\hat{\tau}$ entre 0,97 y 1,16 ms (verdadero 1,0 ms), pero **no la saturación**: α quedó en 10^{-3}, o sea sin saturación, y β absorbió el efecto ($w_{EE} = 7{,}2$ contra 6,4 y $\theta_e = 3{,}07$ contra 2,8). Para un controlador, esto quiere decir que el modelo no anticipa que comandos por encima de 3 pierden efecto.

Para el control, u llega a x solo a través de $\hat{u}$: el grado relativo de P a E es 2. El controlador necesita integrar $\hat{P}$ y $\hat{Q}$ con el comando que aplica, que es lo que hace `IMCExtendido` (`src/neural_ode/closed_loop_ext.py`).

5.3 Latent ODE

`LatentGrayBox` (`src/neural_ode/memory.py`) agrega un estado $z \in \mathbb{R}^2$ sin forma física: $\dot{z} = h_\psi(I, E, z, P, Q)$ y $\dot{x} = f_{WC}(x, u) + g_\phi(I, E, z)$, con $z = 0$ al inicio de cada ventana. Se probó en el escalado sin ruido y dio $R^2 = -0{,}51$ contra el término faltante. No está en las tandas actuales.

6. Cómo se obtienen los parámetros

`scripts/esc_run.py` entrena una configuración. Con datos de 2 s:

1. **Ventanas.** Las trayectorias de entrenamiento (28 escenarios) se parten en ventanas de $W = 10$ muestras (5 ms). Cada una arranca del estado observado (`make_windows`).
2. **Pérdida.** El error cuadrático medio entre la trayectoria predicha por (3) y la observada, sobre todas las ventanas.

3. **Optimización.** Adam durante 1500 épocas, con tasas de 0,05 para los pesos, 0,02 para los demás parámetros físicos y 0,003 para la red, y el gradiente recortado en 10. Después, L-BFGS solo sobre los parámetros físicos.
4. **Selección.** Cada 100 épocas, el modelo se integra libre sobre 7 escenarios de validación, y se guarda el estado con menor NRMSE (`MejorVal`). No se usa el test para elegir nada.
5. **Arranque.** Ignorante, con los diez parámetros en 1,0, o desde el β de un white-box ajustado sobre los mismos datos (`--init-params`).

Funciones: `fit` y `TrainConfig` en `src/neural_ode/graybox_train.py`; `fit_aug` y `AugTrainConfig` en `src/neural_ode/memory.py` para el estado de filtro y la latent ODE, que siguen el mismo esquema sin el paso de L-BFGS.

7. Cuadro comparativo

	white-box	agnóstica	historia	filtro entrenable	forma exacta	estado de filtro
variante en el código	`whitebox`	`B`	`H`	`K`	`S`	`lag`
estado	(I, E)	(I, E)	(I, E)	(I, E)	(I, E)	$(I, E, \hat{P}, \hat{Q})$
entrada de la corrección	no tiene	I, E	I, E, h_k	I, E, Ah_k	no tiene	no tiene
dónde entra u_k	sigmoidea	sigmoidea	sigmoidea y red	sigmoidea y red	sigmoidea, tras saturación	a través de $\hat{u}$
memoria del comando	no	no	K muestras	K muestras	no	un polo, $\hat{\tau}$
afín en u_k dentro de la sigmoidea	sí	sí	no	no	sí, tras invertir la saturación	no; grado relativo 2
parámetros	10	1228	1868 (K = 10)	2124 (n = 8)	13	12
despeje cerrado (ec. 6 del paper)	sí	sí, restando g	no	no	sí	no
qué necesita guardar el controlador	nada	nada	los $K-1$ comandos anteriores	los $K-1$ comandos anteriores	nada	$\hat{P}, \hat{Q}$

8. Consecuencias para el diseño del controlador

Evaluación del modelo dentro del controlador. Todos los modelos son módulos de PyTorch con firma `modelo(x, P, Q) → ẋ`, así que $\partial \dot{x}/\partial u_k$ se obtiene por diferenciación automática. Para las variantes con historia, `_CampoNumpy` (`src/neural_ode/closed_loop_ext.py`) evalúa el mismo campo en numpy, con diferencias del orden de 10^{-8} y unos 200 µs por evaluación. Un checkpoint se carga con `cargar(ruta)`, en `scripts/esc_eval.py`, que reconstruye la variante a partir de los flags guardados.

Invertibilidad. Con historia y filtro entrenable, despejar u_k de $\dot{x} =$ objetivo requiere resolver una ecuación no lineal en (P_k, Q_k) en cada paso. Probado con Newton dentro del IMC, no se encontró un comando mejor que el despeje cerrado en el 40 a 60 % de los pasos. La red no es monótona en u_k en buena parte del rango, y en lazo cerrado el controlador pide comandos de -19 a $+17$, muy fuera de $[0, 2]$. Un controlador que use estos modelos sin invertirlos (MPC, o la estructura IMC clásica con el modelo completo en paralelo) evita ese problema.

Estado de filtro. La inversión es analítica: se invierte la saturación y se compensa el polo. Con la ley $P = \hat{P} + (\hat{\tau}/\tau_f)(P^*_{lag} - \hat{P})$ y $\tau_f = 0{,}2$ ms, el error de seguimiento baja a la mitad respecto del controlador con β verdaderos, pero el comando llega a +73. Con P acotado a $[0, 2]$ y anti-windup, la mejora se mantiene en la referencia gamma de 50 ms y desaparece en la theta-gamma de 2 s.

Valores identificados con σ = 0,01, semilla 0, para quien quiera usar los parámetros directamente:

	w_{EE}	w_{EI}	w_{IE}	w_{II}	τ_e	τ_i	a_e	a_i	θ_e	θ_i
verdaderos	6,40	4,80	6,00	1,20	1,00	2,00	1,20	1,00	2,80	4,00
refractariedad, white-box	5,82	4,90	6,19	1,89	1,12	2,06	1,28	1,05	2,64	3,98
refractariedad, agnóstica con warm start	6,20	4,70	6,16	1,32	1,08	2,04	1,25	0,98	2,69	4,01
refractariedad, forma exacta	6,33	4,77	6,06	1,45	1,01	2,02	1,21	0,98	2,78	4,02
actuador, white-box	8,21	7,30	6,97	2,05	1,41	2,36	1,14	0,91	3,03	4,24
actuador, historia 5 ms con warm start	9,43	7,15	6,99	0,94	1,21	1,98	0,91	0,79	3,56	4,84
actuador, estado de filtro	7,25	5,42	6,26	1,24	1,00	2,00	1,05	0,95	3,07	4,17

9. Archivos y funciones

archivo	qué contiene
`src/neural_ode/dynamics.py`	`GrayBoxWC` (backbone, red, forma exacta), `_FIRMLP` (filtro entrenable)
`src/neural_ode/memory.py`	`LagGrayBox` (estado de filtro), `LatentGrayBox`, `fit_aug`
`src/neural_ode/integrate.py`	`rk4_step`, `rollout`
`src/neural_ode/graybox_train.py`	`TrainConfig`, `VARIANTS`, `build_model`, `make_windows`, `apilar_historia`, `fit`, `MejorVal`, `load_split`
`src/neural_ode/closed_loop.py`	`IMCController` (el controlador del paper), plantas para lazo cerrado
`src/neural_ode/closed_loop_ext.py`	`IMCExtendido`, `_CampoNumpy`, `simulate_closed_loop_ext`, referencia theta-gamma
`src/wilson_cowan/model.py`, `uncertainty.py`	la planta, los estímulos y las perturbaciones
`scripts/esc_run.py`	entrena una configuración y guarda el JSON y el checkpoint
`scripts/esc_eval.py`	`cargar` (reconstruye un modelo desde su checkpoint), métricas